CS171 Assignment 3: Basic Ray Tracing

NAME: JINGXUAN ZHANG
STUDENT NUMBER: 2023533003
EMAIL: ZHANGJX12023

ACM Reference Format:
Name: Jingxuan Zhang, student number: 2023533003, email: zhangjx12023 . 2025. CS171 Assignment 3: Basic Ray Tracing. 1, 1 (November 2025), 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

This report describes my implementation for *CS171 Assignment 3: Basic Ray Tracing*. The goal of the assignment is to implement several core components of a ray tracer, including:

- Ray–triangle intersection.
- Ray–AABB (axis-aligned bounding box) intersection.
- BVH (Bounding Volume Hierarchy) construction and traversal.
- A perfect refraction BSDF for glass-like materials.
- A basic direct illumination integrator with anti-aliasing.
- **[Bonus]** Support for multiple light sources.
- **[Bonus]** Rectangular area lights with soft shadow generation.

In the following sections, I explain how I implemented these features and show key C++ code snippets.

2 Implementation Details

2.1 Ray–AABB Intersection

Requirement. The function `AABB::intersect` in `src/accel.cpp` must test intersection between a ray and an axis-aligned bounding box. It should:

- Compute the entry time t_{in} and exit time t_{out} of the ray with respect to the box.
- Respect the ray's valid parameter interval $[t_{\min}, t_{\max}]$.
- Return `true` if the intersection interval is non-empty, and `false` otherwise.

Implementation. I use the standard slab method. For each coordinate axis, I compute the ray parameter range where the ray is inside that axis-aligned slab using the precomputed safe inverse direction. I maintain a running maximum of entry times and a running minimum of exit times. If after processing all three axes $t_{\text{in}} > t_{\text{out}}$, the ray misses the box.

```
// src/accel.cpp

bool AABB::intersect(const Ray &ray, Float *t_in, Float
  // Use slab method with precomputed inverse direction
  const Vec3f inv_dir = ray.safe_inverse_direction;
  const Vec3f &origin = ray.origin;
  Float tmp_in  = ray.t_min;
  Float tmp_out = ray.t_max;

  for (int i = 0; i < 3; ++i) {
    Float t1 = (low_bnd[i]   - origin[i]) * inv_dir[i];
    Float t2 = (upper_bnd[i] - origin[i]) * inv_dir[i];
    if (t1 > t2) std::swap(t1, t2);
    tmp_in  = std::max(tmp_in,  t1);
    tmp_out = std::min(tmp_out, t2);
  }

  if (tmp_in > tmp_out) return false;
  *t_in  = tmp_in;
  *t_out = tmp_out;
  return true;
}
```

2.2 Ray–Triangle Intersection

Requirement. The function `TriangleIntersect` in `src/accel.cpp` must test intersection between a ray and a single triangle. It should:

- Compute barycentric coordinates (u, v) and the intersection distance t.
- Enforce $u \geq 0$, $v \geq 0$, $u + v \leq 1$, and $t \in [t_{\min}, t_{\max}]$.
- Update the `SurfaceInteraction` using these barycentric weights and clamp `ray.t_max` to the closest hit.

Implementation. I implement the Möller–Trumbore algorithm using double precision internally for robustness. The algorithm:

(1) Compute triangle edges $e_1 = v_1 - v_0$ and $e_2 = v_2 - v_0$.
(2) Compute $s_1 = d \times e_2$ and determinant $\det = e_1 \cdot s_1$.
(3) Reject nearly parallel rays when $|\det|$ is too small.
(4) Compute $s = o - v_0$, then $u = (s \cdot s_1)/\det$, $s_2 = s \times e_1$, $v = (d \cdot s_2)/\det$, $t = (e_2 \cdot s_2)/\det$.

Author's Contact Information: Name: Jingxuan Zhang
student number: 2023533003
email: zhangjx12023.

ACM XXXX-XXXX/2025/11-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

(5) Check barycentric constraints and ray time range and, if valid, update the surface interaction and the ray.

```
// src/accel.cpp

bool TriangleIntersect(Ray & ray, const uint32_t &triangle_index,
    const ref<TriangleMeshResource> &mesh,
    SurfaceInteraction &interaction) {
  using InternalScalarType = Double;
  using InternalVecType    = Vec<InternalScalarType, 3>;

  AssertAllValid(ray.direction, ray.origin);
  AssertAllNormalized(ray.direction);

  const auto &vertices = mesh->vertices;
  const Vec3u v_idx(&mesh->v_indices[3 * triangle_index]);
  assert(v_idx.x < mesh->vertices.size());
  assert(v_idx.y < mesh->vertices.size());
  assert(v_idx.z < mesh->vertices.size());

  InternalVecType dir = Cast<InternalScalarType>(ray.direction);
  InternalVecType v0  = Cast<InternalScalarType>(vertices[v_idx[0]]);
  InternalVecType v1  = Cast<InternalScalarType>(vertices[v_idx[1]]);
  InternalVecType v2  = Cast<InternalScalarType>(vertices[v_idx[2]]);

  InternalVecType e1 = v1 - v0;
  InternalVecType e2 = v2 - v0;

  // s1 = d x e2
  InternalVecType s1 = Cross(dir, e2);
  InternalScalarType det = Dot(e1, s1);
  if (std::abs(det) <= InternalScalarType(1e-9)) return false;
  InternalScalarType invDet = InternalScalarType(1) / det;

  // s = o - v0
  InternalVecType s = Cast<InternalScalarType>(ray.origin) - v0;
  InternalScalarType u = Dot(s, s1) * invDet;
  if (u < 0 || u > 1) return false;

  InternalVecType s2 = Cross(s, e1);
  InternalScalarType v = Dot(dir, s2) * invDet;
  if (v < 0 || u + v > 1) return false;

  InternalScalarType t = Dot(e2, s2) * invDet;
  if (t < ray.t_min || t > ray.t_max) return false;

  // Valid intersection
  CalculateTriangleDifferentials(interaction,
      {static_cast<Float>(1 - u - v),
       static_cast<Float>(u),
       static_cast<Float>(v)},
      mesh, triangle_index);
  AssertNear(interaction.p, ray(t));
  assert(ray.withinTimeRange(t));
  ray.setTimeMax(t);
  return true;
}
```

2.3 BVH Construction

Requirement. In bvh_tree.h, the method BVHTree::build must construct a bounding volume hierarchy over a span of nodes. The method should:

- Build an AABB covering the current span.
- Use stop criteria based on span size and depth to create leaf nodes.
- Otherwise, choose a split dimension (median heuristic) and recursively build left and right subtrees.

Implementation. I implemented a top-down median-split BVH:

- First compute an AABB over the span [span_left, span_right).
- If the span is small (at most 4 primitives) or the depth reaches CUTOFF_DEPTH, I create a leaf node.
- Otherwise, I select the dimension with the largest extent (using ArgMax) and partition nodes along that dimension using std::nth_element.
- Then I recursively build the left and right children and store the resulting internal node.

```
// bvh_tree.h

template <typename _>
typename BVHTree<_>::IndexType BVHTree<_>::build(
    int depth, const IndexType &span_left, const IndexT
  // Stop
  if (span_left >= span_right) return INVALID_INDEX;

  // Compute bounding box for current span
  AABB prebuilt_aabb;
  for (IndexType span_index = span_left; span_index < s
    prebuilt_aabb.unionWith(nodes[span_index].getAABB()

  // Stop criteria: small span or too deep
  if ((span_right - span_left) <= 4 || depth >= CUTOFF_
    InternalNode result(span_left, span_right);
    result.is_leaf = true;
    result.aabb    = prebuilt_aabb;
    internal_nodes.push_back(result);
    return static_cast<IndexType>(internal_nodes.size()
  }

  InternalNode result(span_left, span_right);

  const int dim = ArgMax(prebuilt_aabb.getExtent());
```

```
  IndexType count = span_right - span_left;
  IndexType split = INVALID_INDEX;

  if (hprofile == EHeuristicProfile::EMedianHeuristic) {
    split = span_left + count / 2;

    std::nth_element(
      nodes.begin() + span_left,
      nodes.begin() + split,
      nodes.begin() + span_right,
      [&](const NodeType& a, const NodeType& b) {
        return a.getAABB().getCenter()[dim] < b.getAABB().getCenter()[dim];
      }
    );
  } else if (hprofile == EHeuristicProfile::ESurfaceAreaHeuristic) {
    UNIMPLEMENTED;
  }

  // Build subtrees
  result.left_index  = build(depth + 1, span_left, split);
  result.right_index = build(depth + 1, split, span_right);
  result.aabb        = prebuilt_aabb;

  internal_nodes.push_back(result);
  return static_cast<IndexType>(internal_nodes.size() - 1);
}
```

2.4 Perfect Refraction BSDF

Requirement. In `src/bsdf.cpp`, the method `PerfectRefraction::sample` must model a perfect specular transmission. It should:

- Compute the refracted direction based on Snell's law, for both entering and exiting rays.
- Fall back to perfect reflection under total internal reflection.
- Set `interaction.wi`, return a unit color, and set the PDF to 1 (delta distribution).

Implementation. I use the shading normal and the dot product with `interaction.wo` to determine whether the ray is entering the medium. Based on this, I choose an effective index of refraction and call the helper `Refract`. If refraction fails, I compute the reflected direction via `Reflect`. The PDF is always 1 since this is a delta BSDF.

```
// src/bsdf.cpp

PerfectRefraction::PerfectRefraction(const Properties &props)
    : eta(props.getProperty<Float>("eta", 1.5F)), BSDF(props) {}

Vec3f PerfectRefraction::evaluate(SurfaceInteraction &) const {
  // Delta distribution: no contribution for arbitrary directions
  return {0.0, 0.0, 0.0};
}

Float PerfectRefraction::pdf(SurfaceInteraction &) const {
  return 0;
}

Vec3f PerfectRefraction::sample(
    SurfaceInteraction &interaction, Sampler &sampler,
  // Interface normal (pointing away from surface)
  Vec3f normal = interaction.shading.n;
  // Cosine of the incident angle
  Float cos_theta_i = Dot(normal, interaction.wo);
  // Whether the ray is entering the medium
  bool entering = cos_theta_i > 0;
  // Corrected eta by direction
  Float eta_corrected = entering ? eta : 1.0F / eta;

  // Try to refract; if it fails, use reflection (TIR)
  if (!Refract(interaction.wo, normal, eta_corrected, i
    interaction.wi = Reflect(interaction.wo, normal);
  }
  if (pdf != nullptr) *pdf = 1.0F;
  return Vec3f(1.0);
}
bool PerfectRefraction::isDelta() const {
  return true;
}
```

2.5 Rectangular Area Lights and Soft Shadows (Bonus)

Requirement. The assignment provides an optional task to implement rectangular area lights that produce soft shadows. Such a light should:

- Emit radiance only on the front side of its surface.
- Sample points on the light's area to produce soft penumbrae.
- Provide consistent PDFs for position and direction sampling.

Implementation. I implemented the `AreaLight` class in `src/light.cpp`. The light holds a shape (e.g., a rectangle) and:

- Uses `shape->sample` to pick a surface point for each shading point, which naturally produces soft shadows.
- Returns the constant radiance `radiance` when the outgoing direction is above the light surface ($\cos > 0$).
- Provides cosine-weighted direction sampling and the corresponding PDF.

```
// src/light.cpp
AreaLight::AreaLight(const Properties &props)
    : Light(props),
      radiance(props.getProperty<Vec3f>("radiance", Vec
```

```
void AreaLight::crossConfiguration(const CrossConfiguration &config) {
  clearProperties();
}

Vec3f AreaLight::Le(
    const SurfaceInteraction &interaction, const Vec3f &w) const {
  assert(interaction.light == this);
  assert(interaction.type == ESurfaceInteractionType::ELight);
  return Dot(w, interaction.normal) > 0.0 ? radiance : Vec3f(0.0);
}

Float AreaLight::pdf(const SurfaceInteraction &interaction) const {
  return shape->pdf(interaction);
}

Float AreaLight::pdfDirection(
    const SurfaceInteraction &interaction, const Vec3f &w) const {
  Frame frame(interaction.normal);
  return max(Dot(frame.WorldToLocal(w), DefaultFrameLocalNormal) / PI, 0.0f);
}

SurfaceInteraction AreaLight::sample(
    SurfaceInteraction &interaction, Sampler &sampler) const {
  SurfaceInteraction light_interaction = shape->sample(sampler);

  // Further fill the interaction.
  light_interaction.type = ESurfaceInteractionType::ELight;
  light_interaction.setPrimitive(nullptr, this, nullptr);
  light_interaction.wo = Normalize(interaction.p - light_interaction.p);

  interaction.wi = -light_interaction.wo;

  return light_interaction;
}

SurfaceInteraction AreaLight::sample(Sampler &sampler) const {
  SurfaceInteraction light_interaction = shape->sample(sampler);

  light_interaction.type = ESurfaceInteractionType::ELight;
  light_interaction.setPrimitive(nullptr, this, nullptr);

  return light_interaction;
}

Vec3f AreaLight::sampleDirection(
    const SurfaceInteraction &interaction, Sampler &sampler, Float &pdf) const {
  Frame frame(interaction.normal);

  // Cosine-weighted sampling on hemisphere
  const Vec3f &local_outgoing_direction =
      CosineSampleHemisphere(sampler.get2D());
  pdf = DotCosine(local_outgoing_direction, DefaultFrameLocal
  return frame.LocalToWorld(local_outgoing_direction);
}

Float AreaLight::energy() const {
  return 2 * PI * shape->area() * (radiance.x + radianc
}
```

2.6 Multiple Light Sources (Bonus)

Requirement. Another optional task is to support multiple light sources and accumulate their contributions in direct lighting.

Implementation. In my direct lighting code inside the integrator, I iterate over all lights in `scene->lights`. For each light, I:

- Sample the light using `light->sample` at the current shading point.
- Construct a shadow ray toward the sampled point.
- If the shadow ray is not occluded, evaluate the BSDF and light radiance and accumulate the scaled contribution.

A simplified sketch of the logic is:

```
// Pseudo-code style sketch inside integrator
Vec3f ComputeDirectLighting(const SurfaceInteraction &i
                            Sampler &sampler) const {
  Vec3f Ld(0.0f);
  BSDF *bsdf = isect.bsdf;

  for (auto &light : scene->lights) {
    SurfaceInteraction lightIsect =
        light->sample(const_cast<SurfaceInteraction&>(i
    Vec3f wi = isect.wi; // set by light->sample
    Float pdf = lightIsect.getPdf(EMeasure::ESolidAngle
    if (pdf <= 0) continue;

    // Shadow ray
    Ray shadowRay(isect.p, wi, isect.time, Epsilon,
                  (lightIsect.p - isect.p).Length() - E
    if (scene->occluded(shadowRay)) continue;

    Vec3f f  = bsdf->evaluate(const_cast<SurfaceInterac
    Vec3f Le = light->Le(lightIsect, -wi);

    Ld += f * Le * std::max(0.0f, Dot(wi, isect.shading
  }
  return Ld;
}
```

In my actual code, the function and variable names follow the framework, but the idea is as shown above.

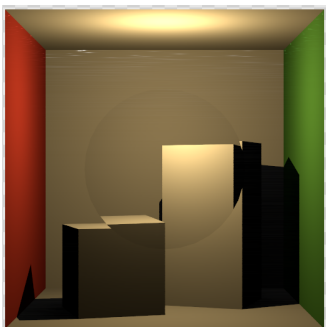

Fig. 1. Final result

2.7 Integrator and Anti-Aliasing

For anti-aliasing, I generate multiple camera rays per pixel by jittering the sample position inside the pixel and averaging the results. The integrator `Li` roughly works as follows:

- Trace a primary ray from the camera.
- Use BVH traversal to find the closest intersection.
- If the BSDF is delta (for example, perfect refraction), call its `sample` function to get the next ray direction and continue.
- If the BSDF is diffuse, compute direct lighting using the multiple-light routine above and terminate the path.

A simplified version of the per-pixel sampling loop is:

```
// Pseudo-code style rendering loop

for (int y = 0; y < height; ++y) {
  for (int x = 0; x < width; ++x) {
    Vec3f L(0.0f);
    for (int s = 0; s < spp; ++s) {
      Vec2f jitter = sampler.get2D();
      Float fx = (x + jitter.x) / Float(width);
      Float fy = (y + jitter.y) / Float(height);
      Ray camRay = camera->generateRay(fx, fy);
      L += integrator->Li(camRay, sampler);
    }
    framebuffer(x, y) = L / Float(spp);
  }
}
```

In my actual implementation, the details follow the framework's interfaces, but the structure is similar.

3 Results

To validate my implementation, I rendered Cornell box scenes provided in the assignment, including:

- `cbox_no_light.json` and `cbox_no_light_refract.json` to test intersection, BVH, and refraction.
- Scenes with rectangular area lights to observe soft shadows and the effect of multiple lights.

The rendered images (with resolution at least 256×256 and at least 4 samples per pixel) show:

- Correct geometric visibility and shadowing, indicating that the ray–triangle, ray–AABB, and BVH traversal implementations are correct.
- Glass panes that refract camera rays as expected from the perfect refraction BSDF.
- Soft penumbrae under area lights, demonstrating that rectangular area light sampling and multiple light support are working as intended.